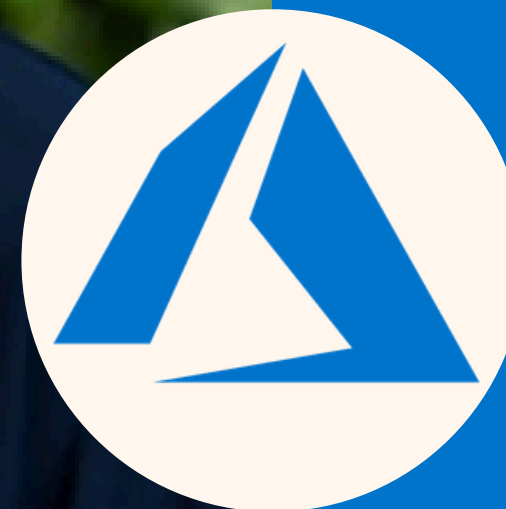


 **GET STARTED TODAY**

LUKE

GINN



**AZURE
CLOUD
MASTERY**



Unit 10 covered custom tool calling. Unit 11 introduces MCP – a standard protocol that eliminates writing custom code for every tool.

- **Review of Unit 10:** You learned to write custom code for each tool – construct HTTP requests, manage API keys, parse responses. This is repetitive.
- **Problem with Custom Code:** Every new tool (Slack, Salesforce, SharePoint) requires custom integration code. Large enterprises have hundreds of tools. Custom code does not scale.
- **What This Unit Covers:** You will learn Model Context Protocol (MCP) – the industry standard "handshake" that lets agents discover and call tools without custom code.

Exam Takeaway: MCP is Microsoft's adopted standard for tool interoperability. The exam tests MCP as the solution to custom integration code.



What Is Model Context Protocol (MCP)?

Model Context Protocol (MCP) is an open standard that defines how agents discover, connect to, and call external tools using a common handshake.

- **MCP Definition:** MCP is a protocol (set of rules) that any agent and any tool can implement. If both support MCP, they can communicate without custom code.
- **The "USB-C for AI" Analogy:** Just as USB-C connects any device to any charger, MCP connects any agent to any MCP-compatible tool without custom drivers.
- **Who Created MCP:** MCP was developed by Anthropic and adopted by Microsoft as the standard for Foundry agents. It is now an industry standard.

Exam Takeaway: MCP is a standard protocol, not a Microsoft-specific feature. The exam tests that MCP eliminates custom integration code.



The MCP handshake is a two-step process: the agent discovers what tools a server offers, then connects and calls them using standard JSON messages.

- **Step 1 – Discovery:** Agent sends ``list_tools`` request to MCP server. Server returns JSON list of available tools, their descriptions, and parameter schemas.
- **Step 2 – Connection:** Agent sends ``call_tool`` request with tool name and parameters. Server executes the tool and returns a standard JSON response.
- **No Custom Code:** The agent does not need to know the tool's API details in advance. The tool describes itself during discovery.

Exam Takeaway: MCP handshake has two phases: discovery (tool lists itself) and execution (agent calls tool). No pre-written integration code needed.



MCP Server – The Tool Provider

An MCP server is a service that exposes one or more tools to agents using the MCP protocol, acting as a bridge between agents and enterprise systems.

- **MCP Server Definition:** The server runs in your infrastructure, connected to your tools (SharePoint, databases, internal APIs). It translates MCP calls to actual API calls.
- **Server Responsibilities:** The server authenticates to downstream tools, handles rate limits, transforms responses to MCP format, and logs all tool calls.
- **Who Runs the Server:** Your team deploys and maintains MCP servers. Microsoft does not provide them. Each enterprise deploys its own servers for its internal tools.

Exam Takeaway: MCP servers are deployed by your team. They bridge MCP agents to your specific enterprise tools. The exam tests this architecture.



MCP Client – The Agent Side

An MCP client is the component inside your agent that speaks the MCP protocol, discovering tools and sending call requests to MCP servers.

- **MCP Client in Agent Framework:** Microsoft Agent Framework includes a built-in MCP client. Your agent uses it without writing protocol code.
- **Client Discovery Flow:** The client sends ``list_tools`` to the server URL you configure. The client caches the tool list for the session.
- **Client Call Flow:** When the agent decides to call a tool, the client sends ``call_tool`` with the tool name and parameters. The client returns the response to the agent.

Exam Takeaway: The Agent Framework provides the MCP client. You configure server URLs. The client handles discovery and calling automatically.



When an agent sends a ``list_tools`` request, the MCP server returns a JSON array describing each available tool, its description, and parameter schema.

- **Response Structure:** ``{"tools": [{"name": "search_sharepoint", "description": "Search documents in SharePoint", "input_schema": {"query": "string", "site_id": "string"}}]}``.
- **Agent Uses Description:** The agent reads the ``description`` field to understand what the tool does. The LLM uses this description to decide when to call the tool.
- **Schema for Parameters:** The ``input_schema`` tells the agent what parameters the tool expects and their data types (string, number, boolean, array).

Exam Takeaway: ``list_tools`` returns tool descriptions and parameter schemas. The agent learns about tools at runtime, not from hardcoded definitions.



To call a tool, the agent sends a `call_tool` JSON message with the tool name and parameters. The server executes the tool and returns a standardized response.

- **Request Structure:** `{"method": "call_tool", "params": {"name": "search_sharepoint", "arguments": {"query": "budget report", "site_id": "finance"}}}`.
- **Response Structure – Success:** `{"content": [{"type": "text", "text": "Found 3 documents: budget_2025.docx, ..."}]}`. The agent receives this text.
- **Response Structure – Error:** `{"isError": true, "content": [{"type": "text", "text": "SharePoint authentication failed"}]}`. Agent sees error and can retry or escalate.

Exam Takeaway: `call_tool` requests and responses have standard JSON formats. The agent handles success and error responses uniformly across all MCP tools.

In Foundry, you configure MCP server endpoints per agent or per project, providing the URLs and authentication credentials for each server.

- **Server Configuration in Foundry:** In agent settings, add an MCP server entry with ``url`` (e.g., ``https://mcp-server.company.com``), ``auth_type`` (bearer token, API key), and ``credentials``.
- **Multiple Servers per Agent:** An agent can connect to multiple MCP servers. The agent discovers tools from all servers and can call any discovered tool.
- **Environment-Specific URLs:** Configure different server URLs for development (``dev-mcp.company.com``), staging, and production. Use environment variables to switch.

Exam Takeaway: MCP servers are configured in Foundry agent settings. One agent can connect to multiple servers. The exam tests configuration structure.



MCP servers authenticate agents using bearer tokens (JWT) or API keys, which are configured in Foundry and injected automatically by the MCP client.

- **Bearer Token Flow:** Agent sends `Authorization: Bearer <token>` header with every MCP request. Foundry obtains the token from Entra ID using the agent's managed identity.
- **API Key Flow:** Configure a static API key in Foundry. The MCP client adds `X-API-Key: <key>` header to all requests to that server.
- **Entra Agent ID Integration:** MCP servers can validate the bearer token against Entra ID, ensuring the agent has permission to call the tool based on its Agent ID.

Exam Takeaway: MCP authentication uses bearer tokens (preferred, with Entra ID) or API keys. The exam tests integration with Entra Agent ID.



MCP vs. Custom Tool Code – When to Use Each

Use MCP when you have many tools or tools that change frequently. Use custom code for simple, stable, single-purpose tools.

- **Use MCP When:** You have 10+ tools, tools are added or removed frequently, or tools are provided by different teams. MCP eliminates per-tool code changes.
- **Use Custom Code When:** You have one or two tools, tools never change, or you need deep optimization (e.g., streaming responses, custom retry logic).
- **Hybrid Approach:** Use MCP for standard enterprise tools (SharePoint, Salesforce, databases). Write custom tools for agent-specific logic (validation, aggregation).

Exam Takeaway: The exam tests MCP vs. custom code selection. MCP for many tools or frequent changes. Custom for simple, stable, few tools.



MCP with SharePoint – Real-World Example

An agent using MCP can search SharePoint documents, read files, and update metadata without writing any SharePoint-specific code.

- **SharePoint MCP Server:** Microsoft provides a pre-built MCP server for SharePoint. Deploy it in your environment. It handles SharePoint authentication and API calls.
- **Agent Discovery:** Agent calls ``list_tools`` on the SharePoint MCP server. Discovers tools: ``search_sharepoint``, ``get_document``, ``update_metadata``.
- **Agent Action:** User asks "Find the budget presentation from last quarter." Agent calls ``search_sharepoint`` with query "budget presentation Q4". Returns document list.

Exam Takeaway: Pre-built MCP servers exist for common Microsoft services. The exam tests that MCP eliminates custom code for these services.



MCP servers can expose local tools like file system operations, command-line scripts, and internal functions that run on your infrastructure.

- **Local File Tool:** MCP server offers ``read_file`` tool with parameter ``path``. Agent can request any file the server has access to. Server reads and returns content.
- **Script Execution Tool:** MCP server offers ``run_script`` tool with parameters ``script_name`` and ``arguments``. Agent can trigger internal analytics or ETL jobs.
- **Security Restriction:** MCP servers should restrict which local paths and scripts are exposed. Do not expose the entire file system. Whitelist allowed operations.

Exam Takeaway: MCP can expose local tools (file system, scripts). Security requires whitelisting. The exam tests understanding of local tool exposure risks.



An agent can compose multiple MCP tool calls, using the output of one tool as the input to another, without writing composition code.

- **Composition Flow:** Agent calls ``search_sharepoint`` to find a document ID. Then calls ``get_document`` with that ID. Then calls ``summarize_text`` on the document content.
- **No Custom Glue Code:** The agent's reasoning handles the composition. The LLM decides the sequence based on tool descriptions and user goal.
- **Error Handling in Chain:** If the first tool fails (no document found), the agent should not call subsequent tools. The agent checks each result before proceeding.

Exam Takeaway: MCP supports tool composition – one tool's output becomes another tool's input. The agent's reasoning handles the chain, not custom code.

Foundry Trace records all MCP operations – discovery requests, tool calls, responses, and errors – as spans in the agent's trace.

- **Discovery Span:** When agent calls ``list_tools``, Foundry Trace creates a span with attributes: ``mcp.server_url``, ``tools_count`` (number discovered), ``duration_ms``.
- **Tool Call Span:** Each ``call_tool`` creates a span with ``tool_name``, ``parameters_json``, ``response_length``, ``status`` (success/error), and ``duration_ms``.
- **Debugging Failures:** If a tool call fails, the span includes ``error_type`` and ``error_message``. Filter traces by ``mcp.server_url`` to find all failures from a specific server.

Exam Takeaway: Instrument MCP calls with Foundry Trace spans. The exam tests adding attributes for server URL, tool name, parameters, and error details.



MCP servers should whitelist (explicitly allow) specific tools for each agent, preventing agents from discovering or calling unauthorized tools.

- **Per-Agent Whitelist:** Configure which tools each agent can discover. Agent A can see ``search_sharepoint`` but not ``delete_file``. Agent B can see both.
- **Whitelist Implementation:** MCP server checks agent's Entra Agent ID against a configuration map. If tool not in whitelist, server omits it from ``list_tools`` response.
- **Call-Time Validation:** Even if an agent somehow knows a tool name, the server rejects ``call_tool`` for tools not on the whitelist. Defense in depth.

Exam Takeaway: MCP servers must whitelist tools per agent identity. The exam tests whitelist configuration and defense-in-depth with call-time validation.



Unit 11 Summary – MCP Standardization

You have learned that MCP standardizes tool discovery and calling, eliminating custom integration code and enabling agents to work with any MCP-compatible tool.

- **MCP Core Concepts:** Two-phase handshake (discovery then calling), MCP server (tool provider), MCP client (agent side), standard JSON message formats.
- **MCP Benefits:** No custom code per tool, tools self-describe, works across any MCP-compatible agent, reduces integration time from days to minutes.
- **Security and Observability:** Whitelist tools per agent identity. Trace all MCP operations with Foundry Trace.
- **Next Unit Preview:** Unit 12 introduces Sequential vs. Concurrent execution patterns – when to run agents one after another vs. simultaneously.

Exam Takeaway: MCP is the industry standard for tool interoperability. Master the two-phase handshake, server vs. client roles, whitelisting, and when to use MCP vs. custom code.

**THANKS
FOR
WATCHING**

